

5. Project Development Phase

5.1 Introduction

The Project Development Phase is the stage in which the proposed system is converted from a design into a functional software application. In the FitBuddy – AI Fitness Plan Generator using Gemini Models project, this phase focuses on implementing the planned frontend, backend, database, and Artificial Intelligence components.

The main objective of development is to create a simple and useful application that can generate personalized fitness and nutrition guidance based on user-provided information. The system combines a web interface with a backend service and Gemini-based AI processing.

- Develop the user interface for entering fitness-related information.
- Create backend APIs for processing requests and generating plans.
- Integrate the Gemini AI model with the application.
- Store required user and application data using a database.
- Connect all modules and verify that data flows correctly between them.

5.2 Development Environment

The application is developed using a modern web development environment. Visual Studio Code is used as the primary development environment because it supports Python, HTML, CSS, JavaScript, API development, and project management in a single workspace.

The backend is implemented using Python and FastAPI. FastAPI provides a lightweight framework for creating REST APIs and handling requests efficiently. The frontend uses HTML, CSS, and JavaScript to provide an interactive user experience.

- Programming language: Python
- Backend framework: FastAPI
- Frontend: HTML, CSS and JavaScript
- Database: SQLite
- AI integration: Gemini API
- Development environment: Visual Studio Code

5.3 Development Approach

Development is carried out module by module. Each component is implemented and tested before it is connected with the other components. This reduces errors and makes debugging easier.

The development process begins with the frontend structure, followed by backend routes, database configuration, AI integration, and finally complete system integration.

5. Project Development Phase – Module Implementation

5.4 Frontend Development

The frontend is the part of FitBuddy that users directly interact with. It provides forms and interface elements through which users can enter information such as fitness goals, age, activity level, preferences, and other relevant details.

The interface is designed to be simple and understandable. Input fields, buttons, labels, and result sections are arranged so that the user can submit information without requiring technical knowledge.

- Create the main application page.
- Add input fields for user fitness information.
- Add a button for generating the fitness plan.
- Create sections for displaying workout and nutrition recommendations.
- Add basic validation to avoid incomplete or invalid input.

5.5 Backend Development

The backend acts as the central processing layer of the application. It receives information from the frontend, validates the data, communicates with the AI service, and sends the generated response back to the user.

FastAPI is used to create backend routes. The backend is organized into separate files so that the application remains easy to understand and maintain.

- Create the main FastAPI application.
- Configure application settings.
- Create API routes for fitness-plan generation.
- Define request and response schemas.
- Handle errors and invalid requests.
- Connect the backend with the database and AI service.

5.6 Database Development

SQLite is used as the database because it is lightweight and suitable for a student-level application. The database can be used to store required application information in a structured manner.

Database models and configuration are created in the backend. The database layer is kept separate from the API routes so that data operations can be managed clearly.

- Configure the SQLite database.
- Create required data models.
- Connect database operations with backend services.

- Test data insertion and retrieval.

5. Project Development Phase – AI Integration

5.7 Gemini AI Integration

One of the major development activities in FitBuddy is the integration of the Gemini AI model. The AI component is responsible for generating personalized fitness and nutrition plans from the information supplied by the user.

The backend prepares a structured prompt containing the user's inputs and sends it to the Gemini API. The generated response is then processed and returned to the frontend for display.

- Collect user-provided fitness information.
- Validate the submitted information.
- Construct an appropriate AI prompt.
- Send the prompt to the Gemini model through the API.
- Receive the generated response.
- Return the result to the application interface.

5.8 Prompt Construction

Prompt construction is important because the quality and usefulness of the generated response depend on how clearly the request is presented to the AI model. The application provides relevant context such as the user's goal, activity level, preferences, and other selected information.

The prompt is designed to request a structured response that can be easily displayed by the application. Separate instructions can be used for workout and nutrition-related outputs.

- Clearly identify the user's fitness goal.
- Include relevant user inputs.
- Request practical and understandable recommendations.
- Ask the model to organize the response into clear sections.
- Avoid unnecessary or unrelated output.

5.9 API Key and Configuration

The Gemini API key is configured securely through application settings or environment variables rather than being directly exposed in the frontend. This helps prevent accidental exposure of credentials.

The backend reads the required configuration when the application starts and uses it when an AI request is made. Proper configuration testing is performed before complete system testing.

- Keep the API key outside frontend source code.
- Configure environment variables correctly.
- Verify that the backend can access the AI configuration.
- Handle API errors without exposing sensitive configuration details.

5.10 Workout and Nutrition Modules

The application can organize AI-generated information into two major areas: workout planning and nutrition guidance. The workout section focuses on suitable exercises and routine structure, while the nutrition section provides general food and meal guidance based on the information supplied by the user.

5. Project Development Phase – Integration and Testing

5.11 System Integration

After individual modules are developed, they are integrated into a single working application. The frontend communicates with the FastAPI backend through API requests. The backend communicates with the database and Gemini AI service, and the final response is returned to the frontend.

Integration testing ensures that information moves correctly from one component to another. Any mismatch between frontend fields, API schemas, backend routes, or AI responses is identified and corrected.

- Connect frontend forms with backend APIs.
- Connect backend routes with database operations.
- Connect backend services with Gemini AI.
- Display generated results on the frontend.
- Test the complete request-response cycle.

5.12 Error Handling

Error handling is included to make the application more reliable. Errors may occur because of missing input, invalid data, API problems, network issues, or configuration errors.

The system should provide understandable messages when an operation cannot be completed. This helps users identify the problem without exposing internal technical information.

- Validate required input fields.
- Handle invalid user data.
- Handle AI API failures.
- Handle database connection problems.
- Display user-friendly error messages.

5.13 Functional Testing

Functional testing is performed to verify whether each feature behaves according to the requirements. Test cases are created for normal input, incomplete input, invalid input, and different combinations of fitness goals.

- Test the home page and input form.
- Test form submission.
- Test API request and response.
- Test AI-generated plan creation.
- Test database operations.
- Test error and validation messages.

5.14 Development Challenges and Solutions

During development, issues may occur while configuring the Python environment, installing dependencies, connecting API services, or running the FastAPI server. These problems are solved by checking configuration files, dependencies, routes, and server logs systematically.

The modular structure of the project makes it easier to locate problems. Separating configuration, models, schemas, routes, templates, and AI-related code also improves maintainability.

5. Project Development Phase – Finalization

5.15 Security and Reliability

Basic security practices are considered during development. Sensitive information such as API credentials should not be placed directly in publicly accessible frontend files. Input validation is also used to reduce unexpected application behavior.

The application is designed so that the backend controls communication with external AI services. This provides a clear separation between the user interface and sensitive service configuration.

- Protect API credentials.
- Validate user inputs.
- Avoid exposing internal errors to users.
- Keep dependencies and configuration organized.
- Test the application before final deployment.

5.16 Final Development Workflow

The completed development workflow starts when the user enters fitness information in the frontend. The data is submitted to the FastAPI backend, validated, and processed. The backend prepares the AI request and communicates with the Gemini model. The generated response is then returned and displayed as a personalized fitness and nutrition plan.

- User enters information.
- Frontend validates and submits the form.
- FastAPI receives the request.
- Backend validates and processes the data.
- Gemini generates the requested plan.
- Backend receives and formats the response.
- Frontend displays the generated plan.

5.17 Development Outcome

The Project Development Phase results in a functional prototype of FitBuddy – AI Fitness Plan Generator using Gemini Models. The major software components are implemented and connected, including the frontend, backend, database, and AI service.

The development phase provides the foundation for later deployment and maintenance. Further improvements can include authentication, improved plan formatting, progress tracking, additional user profiles, stronger validation, and more advanced AI-based personalization.

5.18 Conclusion

The development phase converts the project design into a working application. Through systematic implementation, integration, testing, and error handling, FitBuddy provides a structured platform for generating AI-assisted fitness and nutrition plans. The completed system demonstrates how web

technologies, databases, APIs, and generative AI can be combined to build a practical software application.